

HTB ACADEMY

# Windows Event Logs & Finding Evil

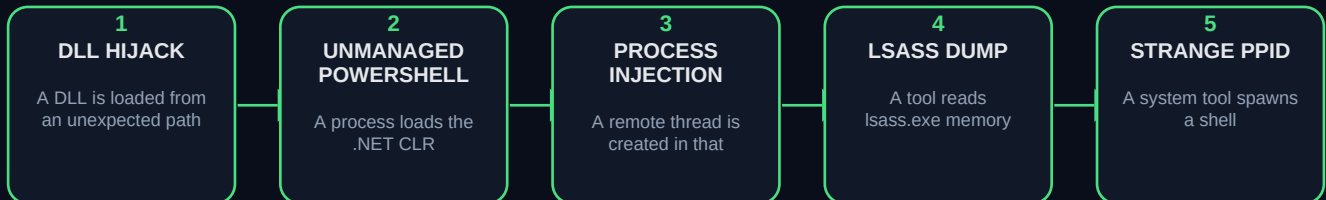
HTB Academy · DFIR / Blue Team Track · Walkthrough

A full attack-chain reconstruction from Windows Event Logs and Sysmon telemetry — DLL hijacking, unmanaged PowerShell execution, process injection, LSASS credential dumping, and PPID spoofing, hunted down one Sysmon event type at a time. Lab answers are partially masked as a hint.

|             |                                                                      |
|-------------|----------------------------------------------------------------------|
| Platform    | HackTheBox Academy                                                   |
| Module      | Windows Event Logs & Finding Evil                                    |
| Tools       | Windows Event Viewer · Sysmon · PowerShell Get-WinEvent · Chainsaw   |
| Analyst     | Chayan Panchal · @tron874                                            |
| Core Skills | Sysmon event analysis, DFIR log hunting, attack-chain reconstruction |

## 1 The Attack Chain

This lab simulates a full intrusion on a single Windows host — from a DLL hijack through to credential dumping and a strange parent-child process relationship. The job is to reconstruct every stage purely from Windows Event Logs and Sysmon telemetry, pre-captured into per-technique .evtx files.



### Key Sysmon event IDs used throughout

| Event ID | Name               | What it detects                                                 |
|----------|--------------------|-----------------------------------------------------------------|
| 1        | ProcessCreate      | New process spawned — captures parent/child relationships       |
| 7        | ImageLoaded        | DLL loaded by a process — detects hijacks and CLR injection     |
| 8        | CreateRemoteThread | Thread created in another process — classic injection indicator |
| 10       | ProcessAccess      | Process opened another process — catches LSASS dumping          |

## 2 Q1 — DLL Hijack

A DLL hijack happens when a legitimate executable loads a malicious DLL from an unexpected location instead of its trusted System32 path — attackers plant a fake DLL where Windows' search order will pick it up first.

Sysmon Event 7 — DismCore.dll loaded from ProgramData, not System32

```
Event ID:      7 (Image Loaded)
Image:         C:\ProgramData\Dism.exe
ImageLoaded:   C:\ProgramData\DismCore.dll
FileVersion:  0.0.0.0
Company:      -
```

**ANSWER (masked)** Dism.exe loading DismCore.dll from ProgramData is a textbook DLL hijack — the real DLL lives in System32, and the fake DLL's placeholder metadata (version 0.0.0.0, no company name) confirms it isn't legitimate.

### 3 Q2 & Q3 — Unmanaged PowerShell + the Injector

"Unmanaged PowerShell" injects .NET/PowerShell code directly into a non-.NET process. The tell is a process with no business running managed code suddenly loading clr.dll, the .NET Common Language Runtime.



Calculator.exe should never load the .NET CLR

```
Event ID:      7 (Image Loaded)
Image:         ...\\Microsoft.WindowsCalculator_...\\Calculator.exe
ImageLoaded:   C:\\Windows\\Microsoft.NET\\Framework64\\v4.0.30319\\clr.dll
```

**ANSWER (masked)** C.....t.....e loading clr.dll is a strong indicator that shellcode was reflectively injected into it to evade detection.



Sysmon Event 8 — who actually put the thread inside Calculator.exe

```
Event ID:      8 (CreateRemoteThread)
SourceImage:   C:\\Windows\\System32\\rundll32.exe
SourcePID:     8364
TargetImage:   ...\\Calculator.exe
TargetPID:     3776
StartAddress:  0x000000253B2180000
```

**ANSWER (masked)** r.....3.....e creating a remote thread inside Calculator.exe is highly anomalous — a legitimate Windows utility being abused to execute code while blending in with normal system activity.

## 4 Q4 & Q5 — LSASS Dump and Post-Dump Login Check

LSASS holds credentials in memory — NTLM hashes, Kerberos tickets, sometimes plaintext passwords. Sysmon Event 10 (ProcessAccess) captures any time one process opens another with read access, which is exactly what a dumping tool does to lsass.exe.

```
PowerShell — filtering Sysmon 10 events targeting lsass.exe

Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=10} |
  Where-Object { $_.Properties[8].Value -eq 'C:\Windows\System32\lsass.exe' } |
  Format-Table TimeCreated, @{n='Source';e={$_.Properties[4].Value}},
    @{n='Access';e={$_.Properties[12].Value}}
```

**ANSWER (masked)** P.....a.....e accessing lsass.exe with PROCESS\_VM\_READ rights is a direct credential-dumping indicator — a legitimate sysadmin tool, but reading LSASS memory on a compromised host is a major red flag.

```
PowerShell — checking for a post-dump login (Security Event 4624)

$dumpTime = Get-Date '2022-04-27 19:20:00'
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624; StartTime=$dumpTime} |
  Where-Object { $_.Properties[5].Value -notlike 'SYSTEM' }
```

**ANSWER** No suspicious interactive/RDP logons were found after the dump — only SYSTEM service logons appeared, indicating the attacker did not use the stolen credentials to log back in during this session. Answer: No.

## 5 Q6 — Strange Parent-Child Process (PPID Spoofing)

PPID spoofing makes a malicious process look like it was launched by a trusted parent. Here, a legitimate error-reporting tool is seen spawning a command shell — something that should never happen during normal operation.

Sysmon Event 1 — WerFault.exe spawning a command shell

Event ID: 1 (ProcessCreate)  
Image: C:\Windows\System32\cmd.exe  
CommandLine: cmd.exe /c whoami  
ParentImage: C:\Windows\System32\WerFault.exe  
CurrentDirectory: C:\ProgramData\

**ANSWER (masked)** W.....e (Windows Error Reporting) spawning cmd.exe is extremely suspicious — WerFault handles crash reports and has no reason to open a shell. The CurrentDirectory pointing back to C:\ProgramData ties this to the same staging location used in the DLL hijack stage.

### Summary table

| # | Question                     | Log location                | Event ID      |
|---|------------------------------|-----------------------------|---------------|
| 1 | DLL hijack process           | C:\Logs\DLLHijack           | Sysmon 7      |
| 2 | Unmanaged PowerShell process | C:\Logs\Powershell<br>IExec | Sysmon 7      |
| 3 | Process injector             | C:\Logs\Powershell<br>IExec | Sysmon 8      |
| 4 | LSASS dumper                 | C:\Logs\Dump                | Sysmon 10     |
| 5 | Post-dump login?             | C:\Logs\Dump                | Security 4624 |
| 6 | Strange PPID process         | C:\Logs\StrangePP<br>ID     | Sysmon 1      |

## 6 Defensive Takeaways

| Finding                              | Mitigation                                                                                 |
|--------------------------------------|--------------------------------------------------------------------------------------------|
| DLL hijack via ProgramData           | Enable Safe DLL Search Mode; restrict DLL loading paths with AppLocker                     |
| Unmanaged PowerShell (CLR injection) | Monitor Sysmon 7 for clr.dll in non-.NET processes; enable AMSI                            |
| Process injection via rundll32       | Alert on Sysmon 8 between unrelated processes; restrict rundll32 via AppLocker             |
| LSASS dumping                        | Enable Credential Guard; restrict SeDebugPrivilege; alert on Sysmon 10 targeting lsass.exe |
| PPID spoofing                        | Monitor Sysmon 1 for system processes spawning shells                                      |

## 7 Quick Reference



Windows Event Logs &amp; Finding Evil — quick reference

|               |                    |                                          |
|---------------|--------------------|------------------------------------------|
| Sysmon 1      | ProcessCreate      | (parent/child relationships)             |
| Sysmon 7      | ImageLoaded        | (DLL loads – hijacks, clr.dll injection) |
| Sysmon 8      | CreateRemoteThread | (process injection)                      |
| Sysmon 10     | ProcessAccess      | (LSASS dumping, credential access)       |
| Security 4624 | Logon              | (post-incident login check)              |

```
Get-WinEvent -FilterHashtable @{LogName='...'; Id=N} | Where-Object {...}
```